
Recherche Opérationnelle et Informatique

Pourquoi c'est important ?

La Recherche Opérationnelle peut sembler éloignée du quotidien d'un informaticien.

Pourtant, les différents prosits de ce bloc suivent exactement le cheminement qu'un développeur ou un administrateur réseau adopte face à un problème technique :

comprendre, évaluer, puis résoudre.

Boucle 1 : Modéliser avec les graphes

Avant d'écrire la moindre ligne de code, il faut représenter le problème. Les graphes sont la structure sous-jacente d'une grande partie des systèmes informatiques.

Graphe → Topologie réseau

Un réseau local est un graphe : les machines sont les sommets, les câbles ou liaisons Wi-Fi sont les arêtes. Quand un administrateur dessine la topologie de son réseau pour diagnostiquer un problème de connectivité, il manipule un graphe sans nécessairement le nommer ainsi.

Coloration de graphe → Attribution de canaux Wi-Fi

Deux bornes Wi-Fi proches qui émettent sur le même canal créent des interférences. Attribuer un canal à chaque borne pour qu'aucune paire de bornes voisines ne partage le même canal, c'est exactement un problème de coloration de graphe. Le nombre chromatique donne le nombre minimal de canaux nécessaires.

Parcours de graphe → Parcours BFS/DFS

Parcourir un graphe en largeur ou en profondeur est l'un des premiers algorithmes enseignés en programmation. On le retrouve dans le crawling d'un site web, dans l'exploration d'un arbre de fichiers, ou encore dans les protocoles de découverte de topologie réseau comme le Spanning Tree Protocol.

Cycle eulérien → Inspection de liens réseau

Un technicien doit tester chaque câble d'un réseau exactement une fois pour vérifier la qualité des liaisons. S'il peut le faire en un seul parcours sans repasser par un câble déjà testé, il suit un cycle eulérien. La condition est simple : chaque noeud du réseau doit avoir un nombre pair de connexions.

Cycle hamiltonien → Tournée de maintenance de serveurs

Un administrateur doit intervenir physiquement sur chaque serveur d'un datacenter exactement une fois, puis revenir à son point de départ. Trouver le trajet le plus court qui passe par chaque machine est un problème de cycle hamiltonien, et c'est un problème NP-difficile : il n'existe pas d'algorithme connu capable de le résoudre efficacement pour un grand nombre de serveurs.

Boucle 2 : Évaluer la difficulté

Une fois le problème modélisé, il faut estimer sa difficulté avant de se lancer dans une implémentation. La théorie de la complexité fournit les outils pour cela.

Classe P → Recherche dans un tableau trié

Un algorithme de recherche dichotomique dans un tableau trié s'exécute en $O(\log n)$: c'est un problème de la classe P. On sait le résoudre efficacement, quelle que soit la taille des données. Le routage par plus court chemin avec Dijkstra, utilisé par le protocole OSPF, relève de la même catégorie.

Classe NP → Vérification de mot de passe

Trouver un mot de passe par force brute peut prendre un temps considérable. En revanche, vérifier qu'un mot de passe donné est correct se fait instantanément. Cette asymétrie entre la difficulté de trouver une solution et la facilité de la vérifier est le principe fondamental de la classe NP, et c'est aussi le fondement de la cryptographie moderne.

Réduction polynomiale → Ramener un bug à un cas connu

Quand un développeur rencontre un bug complexe, son premier réflexe est de le ramener à un problème déjà documenté : reproduire le cas minimal, identifier le pattern, puis appliquer la solution connue. C'est exactement le principe d'une réduction : transformer un problème inconnu en un problème que l'on sait déjà traiter.

NP-complétude → Routage multi-contraintes

Acheminer un flux réseau en respectant simultanément des contraintes de latence, de bande passante et de fiabilité est un problème NP-complet. C'est pour cette raison que les protocoles de routage réels utilisent des heuristiques plutôt qu'un calcul exact : ils acceptent une solution suffisamment bonne plutôt que de chercher la solution parfaite.

Force brute vs algorithme efficace → Requête SQL avec ou sans index

Interroger une table de plusieurs millions de lignes sans index oblige le moteur de base de données à parcourir chaque ligne : c'est un parcours en $O(n)$. Ajouter un index ramène la recherche à $O(\log n)$. La différence entre les deux est exactement ce que mesure la complexité algorithmique : un même problème peut être facile ou coûteux selon la méthode choisie.

Boucle 3 : Choisir la bonne stratégie de résolution

Quand le problème est identifié comme difficile, il reste à trouver une méthode de résolution adaptée. Chaque approche de la Recherche Opérationnelle a son équivalent dans la pratique informatique.

Contraintes et fonction objectif → Dimensionnement d'infrastructure

Minimiser le coût d'un hébergement cloud tout en garantissant un temps de réponse inférieur à 200 ms et une disponibilité de 99,9 % : c'est un problème d'optimisation sous contraintes. La fonction objectif est le coût, les contraintes sont la latence et la disponibilité. C'est exactement la structure d'un programme linéaire.

Simplexe → Algorithme glouton

Le Simplexe progresse de sommet en sommet du polytope des solutions en choisissant à chaque étape le voisin qui améliore le plus la fonction objectif. Ce comportement rappelle les algorithmes gloutons en programmation : à chaque étape, on fait le choix localement optimal. La différence est que le Simplexe garantit l'optimalité dans le cas continu, ce qu'un algorithme glouton ne garantit pas en général.

Branch & Bound → Backtracking et élagage

Un solveur de Sudoku explore les possibilités case par case et abandonne une piste dès qu'une contradiction apparaît : c'est du backtracking avec élagage. Le Branch & Bound fonctionne de la même façon : il construit un arbre de recherche et coupe les branches dont on peut prouver qu'elles ne mèneront pas à une meilleure solution que celle déjà connue.

Programmation dynamique → Cache et mémorisation

Le calcul récursif de Fibonacci recalcule les mêmes valeurs des dizaines de fois. Stocker chaque résultat intermédiaire dans un tableau pour ne le calculer qu'une seule fois, c'est de la mémorisation, et c'est le principe exact de la programmation dynamique. On retrouve la même logique dans les systèmes de cache : ne jamais recalculer ou retélécharger ce que l'on connaît déjà.

PLNR vs PLNE → Bande passante vs serveurs physiques

Allouer de la bande passante en continu entre plusieurs flux est un problème continu : on peut attribuer 12,7 Mbps à un flux. En revanche, affecter des serveurs physiques à des services est un problème discret : on ne peut pas attribuer 2,3 serveurs. Ce passage du continu au discret change radicalement la difficulté du problème, et c'est précisément la différence entre un PLNR et un PLNE.

Boucle 4 : Résoudre à grande échelle avec les méta-heuristiques

Quand les méthodes exactes deviennent trop coûteuses en temps de calcul, il faut accepter de chercher une bonne solution plutôt que la solution parfaite. Les méta-heuristiques explorent intelligemment l'espace des solutions, et leurs principes se retrouvent dans de nombreux mécanismes informatiques.

Optimum local vs optimum global → Descente de gradient en Machine Learning

Entraîner un réseau de neurones revient à chercher le minimum d'une fonction de coût. La descente de gradient peut rester bloquée dans un minimum local, une vallée qui semble être le point le plus bas mais qui ne l'est pas à l'échelle globale. Les méta-heuristiques affrontent exactement le même problème : trouver des mécanismes pour s'échapper des optima locaux et continuer à explorer.

Exploration vs exploitation → Stratégie de cache

Un système de cache doit en permanence arbitrer : continuer à servir les données déjà en cache qui fonctionnent bien, ou tester de nouvelles stratégies de pré-chargement qui pourraient être plus performantes. Cet équilibre entre exploiter ce qui marche et explorer ce qui pourrait mieux marcher est le dilemme fondamental de toute méta-heuristique.

Recuit simulé → Contrôle de congestion TCP

TCP démarre avec une fenêtre d'émission large, acceptant beaucoup de variabilité, puis réduit progressivement son agressivité pour converger vers un débit stable. Le recuit simulé suit la même logique : au départ, il accepte des solutions dégradées pour explorer largement l'espace, puis il réduit progressivement cette tolérance pour converger vers une bonne solution. Dans les deux cas, on commence par de grands sauts puis on affine.

Algorithme génétique → Tests A/B et déploiement progressif

Lors d'un déploiement en production, on teste plusieurs variantes en parallèle, on mesure leurs performances, on conserve les meilleures et on les combine pour créer la version suivante. C'est le principe d'un algorithme génétique : une population de solutions évolue par sélection, croisement et mutation, génération après génération, jusqu'à converger vers une solution performante.

Recherche tabou → Mécanismes anti-boucle en routage

Les protocoles de routage comme RIP utilisent des mécanismes tels que le split horizon ou le route poisoning pour éviter qu'un paquet ne tourne en boucle entre deux routeurs. La recherche tabou applique le même principe : elle maintient une liste des solutions récemment visitées et s'interdit d'y revenir, ce qui l'empêche de tourner en rond et la force à explorer de nouvelles régions de l'espace des solutions.

Colonies de fourmis → Protocoles de routage distribués

Dans un réseau, chaque routeur prend ses décisions localement en se basant sur les informations partagées par ses voisins, et pourtant le réseau converge globalement vers des chemins efficaces. Les algorithmes de colonies de fourmis fonctionnent de la même manière : chaque agent dépose une trace locale, et c'est l'accumulation de ces traces individuelles qui fait émerger collectivement les meilleurs chemins.